07. OSI Model

```
┌──────────────────────────────────────────────────────────────────┐
│                          OSI MODEL                               │
│      Open Systems Interconnection Reference Model                 │
│                                                                   │
│   Created by: ISO (International Organization for Standards)    │
│   Year: 1984                                                      │
│   Purpose: Standardize network communication                     │
└──────────────────────────────────────────────────────────────────┘
```

The OSI model is a conceptual framework that describes how data moves from one computer to another across a network.

7 Layers

```
Layer   Name            Data Unit      Key Function
═══════════════════════════════════════════════════════════════════

  7     Application     Data           User interface & services
        ▲
  6     Presentation    Data           Encryption, Compression, Format
        │
  5     Session         Data           Session management
        │
─────── │ ──────────────────────────────────────────────────────
        │   (Upper Layers - Host Layers - SOFTWARE)
        │   (Lower Layers - Media Layers - HARDWARE/SOFTWARE)
─────── │ ──────────────────────────────────────────────────────
        │
  4     Transport       Segment        End-to-end delivery
        │
  3     Network         Packet         Routing & Logical addressing
        │
  2     Data Link       Frame          Physical addressing & Error detection
        │
  1     Physical        Bits           Physical transmission
        ▼

═══════════════════════════════════════════════════════════════════
```

Cheat sheet

Layer	Name	PDU	Address	Devices	Protocols	Key Fu
7	Application	Data	N/A	Gateway, Proxy	HTTP, HTTPS, FTP, SMTP, DNS, DHCP, SSH, Telnet	User in networl
6	Presentation	Data	N/A	Gateway	SSL/TLS, JPEG, MPEG, GIF, ASCII, EBCDIC	Encryp Compre Transla
5	Session	Data	Session ID	Gateway	NetBIOS, RPC, PPTP, SCP	Sessio manag Authen session establis
4	Transport	Segment/Datagram	Port Number	Firewall, Load Balancer	TCP, UDP	End-to- Flow co
3	Network	Packet	IP Address	Router, L3 Switch	IP, ICMP, ARP, OSPF, BGP, RIP	Logical Routing
2	Data Link	Frame	MAC Address	Switch, Bridge, NIC	Ethernet, WiFi, PPP, ARP	Physica Framin
1	Physical	Bits	N/A	Hub, Repeater, Cables, NIC	RS-232, DSL, ISDN, USB	Bit tran

Layer 7: Application Layer

The layer where the USER and the APPLICATION interact directly. This is what you see in your browser, email client, etc.

Protocols at Layer 7

```
┌──────────────────────────────────────────────────────────────────────┐
│                        APPLICATION PROTOCOLS                         │
├──────────────┬──────────────┬──────────────────────────────────────┤
│  Protocol    │    Port      │              Purpose                 │
├──────────────┼──────────────┼──────────────────────────────────────┤
│  HTTP        │    80        │  Web browsing (unencrypted)          │
│  HTTPS       │    443       │  Web browsing (encrypted/TLS)        │
│  FTP         │   20, 21     │  File transfer                       │
│  SFTP        │    22        │  Secure file transfer                │
│  SSH         │    22        │  Secure remote access                │
│  TELNET      │    23        │  Remote access (unencrypted)         │
│  SMTP        │    25        │  Email sending                       │
│  DNS         │    53        │  Domain name resolution              │
│  DHCP        │   67, 68     │  IP address assignment               │
│  POP3        │    110       │  Email retrieval                     │
│  IMAP        │    143       │  Email retrieval (advanced)          │
│  SNMP        │   161, 162   │  Network management                  │
│  LDAP        │    389       │  Directory services                  │
│  RDP         │    3389      │  Remote desktop                      │
└──────────────┴──────────────┴──────────────────────────────────────┘
```

Request Flow

```
USER REQUEST FLOW:

    ┌──────────┐         ┌──────────────┐         ┌──────────┐
    │  User    │ ──────► │ Application  │ ──────► │  Network │
    │ (Types   │         │  (Browser/   │         │ (HTTP    │
    │  URL)    │         │   Client)    │         │ Request) │
    └──────────┘         └──────────────┘         └──────────┘

Example: User types "www.google.com"

    1. Browser (Application) receives request
    2. Creates HTTP/HTTPS request
    3. Passes to lower layers for transmission
```

WAF works at Layer 7

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    LAYER 7 SECURITY MEASURES                             │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  WEB APPLICATION FIREWALL (WAF)                                          │
│  ──────────────────────────────                                          │
│                                                                          │
│   ┌──────────┐      ┌──────────┐      ┌──────────────┐                  │
│   │ Internet │──────►│   WAF    │──────►│ Web Server   │                  │
│   │ Traffic  │      │ (Filter) │      │ (Protected)  │                  │
│   └──────────┘      └──────────┘      └──────────────┘                  │
│                          │                                               │
│                          ▼                                               │
│                 ┌──────────────────┐                                     │
│                 │ Blocks:          │                                     │
│                 │ - SQL Injection  │                                     │
│                 │ - XSS            │                                     │
│                 │ - CSRF           │                                     │
│                 │ - Bot traffic    │                                     │
│                 └──────────────────┘                                     │
│                                                                          │
│  INPUT VALIDATION                                                        │
│  ────────────────                                                        │
│  - Whitelist allowed characters                                          │
│  - Sanitize all user inputs                                              │
│  - Use parameterized queries                                             │
│                                                                          │
│  SECURE PROTOCOLS                                                        │
│  ────────────────                                                        │
│   ┌──────────────────┬──────────────────────────────────────┐            │
│   │ Instead of       │ Use                                  │            │
│   ├──────────────────┼──────────────────────────────────────┤            │
│   │ HTTP             │ HTTPS                                │            │
│   │ FTP              │ SFTP/FTPS                            │            │
│   │ Telnet           │ SSH                                  │            │
│   │ SMTP (plain)     │ SMTP with TLS                        │            │
│   └──────────────────┴──────────────────────────────────────┘            │
│                                                                          │
│  DNSSEC                                                                  │
│  ──────                                                                  │
│  - Cryptographically signs DNS records                                   │
```

```
│  - Prevents DNS spoofing/cache poisoning                              │
│                                                                       │
└───────────────────────────────────────────────────────────────────────┘
```

DNS Query works at Layer 7

DNS query works at layer 7 but use layer 4 (Transport Layer Protocol) ex=UDP/TCP

more at :-
[08. DNS Query]

Layer 6: Presentation Layer

Layer 6 is responsible for how data is presented/formatted so that both sides can understand it.

Translation, Encryption, Compression

It makes sure data is in usable format for Application layer

Data Formatting & Translation

Converts data into a standard format.
Example:

- JSON
- XML
- ASCII / Unicode

```
┌───────────────────────────────────────────────────────────────────────┐
│                     DATA TRANSLATION                                  │
├───────────────────────────────────────────────────────────────────────┤
│                                                                       │
│  Character Encoding:                                                  │
│  ┌──────────────┬──────────────────────────────────────────────────┐ │
│  │ ASCII        │ 7-bit, 128 characters (English only)             │ │
│  │ Extended     │ 8-bit, 256 characters                            │ │
│  │ ASCII        │                                                  │ │
│  │ Unicode      │ Universal character set                          │ │
│  │ UTF-8        │ Variable-length Unicode encoding (1-4 bytes)     │ │
│  │ UTF-16       │ Fixed 2 or 4 bytes per character                 │ │
│  └──────────────┴──────────────────────────────────────────────────┘ │
│                                                                       │
```

```
│  Example - Letter 'A':                                          │
│  ┌──────────────┬──────────────────────────────────────┐        │
│  │ ASCII        │ 65 (decimal) = 01000001 (binary)     │        │
│  │ UTF-8        │ 0x41                                 │        │
│  │ UTF-16       │ 0x0041                               │        │
│  └──────────────┴──────────────────────────────────────┘        │
│                                                                  │
│  Data Serialization Formats:                                     │
│  ┌──────────────┬──────────────────────────────────────────────┐ │
│  │ JSON         │ {"name": "John", "age": 30}                  │ │
│  │ XML          │ <person><name>John</name><age>30</age></person> │ │
│  │ YAML         │ name: John\n age: 30                         │ │
│  │ Binary       │ Protocol Buffers, MessagePack                │ │
│  └──────────────┴──────────────────────────────────────────────┘ │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

TLS/SSL for encryption only handshake done on layer 5(session layer): (HTTPS uses TLS to encrypt data)

```
+-------------------------------------------------------------+
|                     WHAT IS SSL/TLS?                       |
+-------------------------------------------------------------+
|                                                             |
|   SSL = Secure Sockets Layer (Old name)                     |
|   TLS = Transport Layer Security (New name)                 |
|                                                             |
|   SIMPLE MEANING:                                           |
|   A security protocol that protects data                    |
|   traveling between your browser and website                |
|                                                             |
|   LIKE:                                                     |
|   Sending a letter in a LOCKED BOX instead of               |
|   an open postcard                                          |
|                                                             |
+-------------------------------------------------------------+
```

Encryption using TLS

```
┌──────────────────────────────────────────────────────────────────┐
│                          ENCRYPTION                              │
├──────────────────────────────────────────────────────────────────┤
```

```
SYMMETRIC ENCRYPTION (Same key for encrypt/decrypt)
___________________________________________________

┌──────────┐     ┌──────────────┐     ┌────────────────┐
│ Plaintext │──►│  Encrypt     │──►│  Ciphertext    │
│  "Hello"  │     │  (Key: K)    │     │  "xY7#@..."    │
└──────────┘     └──────────────┘     └────────────────┘

┌────────────────┐     ┌──────────────┐     ┌──────────┐
│  Ciphertext    │──►│  Decrypt     │──►│ Plaintext │
│  "xY7#@..."    │     │  (Key: K)    │     │  "Hello"  │
└────────────────┘     └──────────────┘     └──────────┘

Algorithms:
┌──────────────┬──────────┬──────────────────────────────┐
│ Algorithm    │ Key Size │ Notes                        │
├──────────────┼──────────┼──────────────────────────────┤
│ AES          │ 128/192/ │ Current standard, very secure │
│              │ 256 bits │                              │
│ 3DES         │ 168 bits │ Legacy, being phased out     │
│ DES          │ 56 bits  │ Obsolete, easily crackable   │
│ Blowfish     │ 32-448   │ Fast, used in bcrypt         │
│ ChaCha20     │ 256 bits │ Modern, used in TLS 1.3      │
└──────────────┴──────────┴──────────────────────────────┘

ASYMMETRIC ENCRYPTION (Public/Private key pair)
_______________________________________________

┌──────────────────────────────────────────────────────┐
│                                                      │
│   Sender                              Receiver       │
│   ┌───────┐                           ┌───────┐      │
│   │Encrypt│◄── Receiver's ──────────►│Decrypt│      │
│   │with   │    Public Key             │with   │      │
│   │PubKey │                           │PriKey │      │
│   └───┬───┘                           └───▲───┘      │
│       │                                   │          │
│       └──────── Ciphertext ───────────────┘          │
│                                                      │
└──────────────────────────────────────────────────────┘

Algorithms:
┌──────────────┬──────────┬──────────────────────────────┐
```

```
│   │ Algorithm     │ Key Size │ Use Case                              │ │
│   ├────────────────┼──────────┼───────────────────────────────────────┤ │
│   │ RSA            │ 2048+    │ Key exchange, digital signatures      │ │
│   │ ECC            │ 256+     │ Mobile, IoT (smaller keys)            │ │
│   │ Diffie-        │ 2048+    │ Key exchange                          │ │
│   │ Hellman        │          │                                       │ │
│   │ ECDH           │ 256+     │ Modern key exchange                   │ │
│   └────────────────┴──────────┴───────────────────────────────────────┘ │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

Data Compression

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        DATA COMPRESSION                                  │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  LOSSLESS (Perfect reconstruction)                                       │
│  ─────────────────────────────────                                       │
│   ┌────────────────┬──────────────────────────────────────────────────┐ │
│   │ Algorithm      │ Use Case                                         │ │
│   ├────────────────┼──────────────────────────────────────────────────┤ │
│   │ DEFLATE        │ ZIP files, HTTP compression (gzip)               │ │
│   │ LZ77/LZ78      │ Foundation for many algorithms                   │ │
│   │ Huffman        │ Frequently used in combination with others       │ │
│   │ Brotli         │ Modern web compression                           │ │
│   └────────────────┴──────────────────────────────────────────────────┘ │
│                                                                          │
│  LOSSY (Some data lost, smaller size)                                    │
│  ─────────────────────────────────────                                   │
│   ┌────────────────┬──────────────────────────────────────────────────┐ │
│   │ Format         │ Use Case                                         │ │
│   ├────────────────┼──────────────────────────────────────────────────┤ │
│   │ JPEG           │ Photos, images                                   │ │
│   │ MP3/AAC        │ Audio                                            │ │
│   │ H.264/H.265    │ Video                                            │ │
│   └────────────────┴──────────────────────────────────────────────────┘ │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

Layer 5: Session Layer

Managing communication sessions (dialogue control) Establishes, maintains, and terminates sessions between applications.

Layer 5 is responsible for creating, managing, and ending sessions between two devices/applications.

- **Session Management**
- **Synchronization:** Adds checkpoints during long data transfers.
  If communication fails, it resumes from the last checkpoint instead of restarting completely.

```
┌──────────────────────────────────────────────────────────────────┐
│                      SESSION LAYER                               │
├──────────────────────────────────────────────────────────────────┤
│  Purpose: Manage sessions/connections between applications       │
│  Data Unit: Data                                                 │
│  Nickname: "Dialog Controller"                                   │
│                                                                  │
│  Main Functions:                                                 │
│   ┌────────────────────┬──────────────────────────────────────┐  │
│   │ Session            │ Establish, maintain, terminate       │  │
│   │ Establishment      │ communication sessions               │  │
│   │ Dialog Control     │ Manage turn-taking in communication  │  │
│   │ Synchronization    │ Add checkpoints for recovery         │  │
│   │ Authentication     │ Verify identity at session start     │  │
│   └────────────────────┴──────────────────────────────────────┘  │
└──────────────────────────────────────────────────────────────────┘
```

Session Management Process

```
┌──────────────────────────────────────────────────────────────────┐
│                    SESSION LIFECYCLE                             │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│   ┌──────────────────────────────────────────────────────────┐   │
│   │                                                          │   │
│   │   1. SESSION ESTABLISHMENT                               │   │
│   │   ────────────────────────                               │   │
│   │   Client                                    Server       │   │
│   │     │                                         │          │   │
│   │     │─── Connection Request ─────────────────►│          │   │
│   │     │                                         │          │   │
```

```
│  │     │◄── Authentication Challenge ──────────│              │  │
│  │     │                                       │              │  │
│  │     │─── Credentials ──────────────────────►│              │  │
│  │     │                                       │              │  │
│  │     │◄── Session Established ───────────────│              │  │
│  │     │    (Session ID assigned)              │              │  │
│  │                                                            │  │
│  │   2. SESSION MAINTENANCE                                   │  │
│  │   ──────────────────────────                               │  │
│  │     │◄═══ Data Exchange ═══════════════════►│              │  │
│  │     │                                       │              │  │
│  │     │─── Keep-Alive ───────────────────────►│              │  │
│  │     │◄── ACK ───────────────────────────────│              │  │
│  │     │                                       │              │  │
│  │     │  [Synchronization Points/Checkpoints] │              │  │
│  │                                                            │  │
│  │   3. SESSION TERMINATION                                   │  │
│  │   ──────────────────────────                               │  │
│  │     │                                       │              │  │
│  │     │─── Close Request ────────────────────►│              │  │
│  │     │◄── Close Acknowledgment ──────────────│              │  │
│  │     │                                       │              │  │
│  │   [Session terminated, resources released]                 │  │
│  │                                                            │  │
│  └────────────────────────────────────────────────────────────┘  │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

SSL/TLS Handshake

```
+=================================================================+
|           SSL/TLS HANDSHAKE AT LAYER 5 (SESSION LAYER)          |
+=================================================================+

   CLIENT                                                SERVER
   (Browser)                                            (Website)
      |                                                     |
      |                                                     |
      |  ╔═══════════════════════════════════════════════╗  |
      |  ╠         PHASE 1: SESSION START               ╣  |
      |  ╚═══════════════════════════════════════════════╝  |
      |                                                     |
      |                                                     |
```

```
│  STEP 1: CLIENT HELLO                                  │
│  ┌──────────────────────────────────────────────┐      │
│  │  "Hi! I want to start secure session"   │      │
│  │                                              │      │
│  │  I support:                                  │      │
│  │  • TLS Version: 1.2, 1.3                     │      │
│  │  • Ciphers: AES-256, ChaCha20                │      │
│  │  • My Random Number: abc123...               │      │
│  │  • Session ID: (empty or old)                │      │
│  └──────────────────────────────────────────────┘      │
│──────────────────────────────────────────────────────>│
│                                                        │
│                                                        │
│  STEP 2: SERVER HELLO                                  │
│     ┌──────────────────────────────────────────────┐   │
│     │  "Hello! Let's create session"          │   │
│     │                                              │   │
│     │  I choose:                                   │   │
│     │  • TLS Version: 1.3                          │   │
│     │  • Cipher: AES-256-GCM                       │   │
│     │  • My Random Number: xyz789...               │   │
│     │  • Session ID: SESSION123                    │   │
│     └──────────────────────────────────────────────┘   │
│<──────────────────────────────────────────────────────│
│                                                        │
│                                                        │
│  ╔══════════════════════════════════════════════╗      │
│  ╠═══════         PHASE 2: AUTHENTICATION      ═══╣      │
│  ╚══════════════════════════════════════════════╝      │
│                                                        │
│                                                        │
│  STEP 3: SERVER CERTIFICATE                            │
│     ┌──────────────────────────────────────────────┐   │
│     │  "Here is my ID card (Certificate)"     │   │
│     │                                              │   │
│     │  ┌──────────────────────────────────────┐   │   │
│     │  │  CERTIFICATE                         │   │   │
│     │  │  ────────────                        │   │   │
│     │  │  Domain: google.com                  │   │   │
│     │  │  Owner: Google LLC                   │   │   │
│     │  │  Issuer: DigiCert CA                 │   │   │
│     │  │  Valid: 2024-01-01 to 2025-01-01     │   │   │
│     │  │  Public Key: MIIBIjAN...             │   │   │
│     │  │  Signature: 4f7a8b2c...              │   │   │
```

```
│    │  └──────────────────────────────────────┘   │  │
│    └──────────────────────────────────────────┘   │
│<──────────────────────────────────────────────────│
│                                                   │
│                                                   │
│  STEP 4: CLIENT VERIFIES CERTIFICATE              │
│  ┌──────────────────────────────────────────┐     │
│  │  Client checks:                          │     │
│  │                                          │     │
│  │  ✓ Is certificate expired? NO            │     │
│  │  ✓ Is domain correct? YES (google.com)   │     │
│  │  ✓ Is CA trusted? YES (DigiCert)         │     │
│  │  ✓ Is signature valid? YES               │     │
│  │                                          │     │
│  │  RESULT: Certificate is TRUSTED!         │     │
│  └──────────────────────────────────────────┘     │
│        (Happens locally on client)                │
│                                                   │
│                                                   │
│  ╔═══════════════════════════════════════════╗    │
│  ╠═══╗        PHASE 3: KEY EXCHANGE          ╠═══╗
│  ╚═══╝═══════════════════════════════════════╝    │
│                                                   │
│                                                   │
│  STEP 5: CLIENT KEY EXCHANGE                      │
│  ┌──────────────────────────────────────────┐     │
│  │  Client generates:                       │     │
│  │  • Pre-Master Secret (random 48 bytes)   │     │
│  │                                          │     │
│  │  Client encrypts Pre-Master Secret       │     │
│  │  with Server's PUBLIC KEY                │     │
│  │                                          │     │
│  │  Only server can decrypt this!           │     │
│  │  (Because only server has PRIVATE KEY)   │     │
│  └──────────────────────────────────────────┘     │
│──────────────────────────────────────────────────>│
│                                                   │
│                                                   │
│  STEP 6: SERVER DECRYPTS                          │
│    ┌──────────────────────────────────────────┐   │
│    │  Server decrypts Pre-Master Secret      │  │
│    │  using its PRIVATE KEY                  │  │
│    │                                          │  │
│    │  Now server has:                        │  │
```

```
|     |  • Client Random                          |  |
|     |  • Server Random                          |  |
|     |  • Pre-Master Secret                      |  |
|     └──────────────────────────────────────────┘  |
|                                                   |
|                                                   |
|  ╔═══════════════════════════════════════════════╗ |
|  ║         PHASE 4: SESSION KEY CREATION         ╕ |
|  ╚═══════════════════════════════════════════════╝ |
|                                                   |
|                                                   |
|  STEP 7: BOTH CREATE SESSION KEY                  |
|                                                   |
|  ┌──────────────────────────────────────────────┐ |
|  │                                              │ |
|  │   CLIENT SIDE:          SERVER SIDE:         │ |
|  │                                              │ |
|  │   Client Random    +    Client Random        │ |
|  │        +                    +                │ |
|  │   Server Random    +    Server Random        │ |
|  │        +                    +                │ |
|  │   Pre-Master       +    Pre-Master           │ |
|  │        =                    =                │ |
|  │   SESSION KEY      =    SESSION KEY          │ |
|  │                                              │ |
|  │        (SAME KEY ON BOTH SIDES!)             │ |
|  │                                              │ |
|  └──────────────────────────────────────────────┘ |
|                                                   |
|                                                   |
|  ╔═══════════════════════════════════════════════╗ |
|  ║         PHASE 5: SESSION ESTABLISHED          ╕ |
|  ╚═══════════════════════════════════════════════╝ |
|                                                   |
|                                                   |
|  STEP 8: CHANGE CIPHER SPEC                       |
|  ┌──────────────────────────────────────────┐     |
|  │  "I'm switching to encrypted mode now"   │     |
|  └──────────────────────────────────────────┘     |
|──────────────────────────────────────────────────>|
|                                                   |
|     ┌──────────────────────────────────────────┐  |
|     │  "Me too! Switching to encrypted mode"   │  |
|     └──────────────────────────────────────────┘  |
```

```
      │<──────────────────────────────────────────────────────────│
      │                                                           │
      │                                                           │
      │  STEP 9: FINISHED                                         │
      │  ┌──────────────────────────────────────────────┐         │
      │  │  "Handshake complete! Session ready!"         │         │
      │  └──────────────────────────────────────────────┘         │
      │──────────────────────────────────────────────────────────>│
      │                                                           │
      │                                                           │
      │  ╔═══════════════════════════════════════════════════╗    │
      │  ║    SECURE SESSION ESTABLISHED!                    ║    │
      │  ║    (Now Layer 6 takes over for                    ║    │
      │  ║     encryption/decryption)                        ║    │
      │  ╚═══════════════════════════════════════════════════╝    │
      │                                                           │
      │  ═══════ ENCRYPTED COMMUNICATION ═══════                  │
      │<──────────────────────────────────────────────────────────│
      │                                                           │
```

Layer 4: Transport Layer

End-to-end reliable data transfer & flow control This is where TCP and UDP live.

```
┌──────────────────────────────────────────────────────────────────────┐
│                        TRANSPORT LAYER                               │
├──────────────────────────────────────────────────────────────────────┤
│  PDU: SEGMENT (TCP) / DATAGRAM (UDP)                                 │
│  Devices: Firewalls (L4), Load Balancers                             │
│  Addressing: Port Numbers (0-65535)                                  │
│  Functions:                                                          │
│    • End-to-End Communication                                        │
│    • Segmentation and Reassembly                                     │
│    • Flow Control                                                    │
│    • Error Detection and Recovery                                    │
│    • Multiplexing/Demultiplexing                                     │
└──────────────────────────────────────────────────────────────────────┘
```

```
    ┌──────────────────────────────────────────────────────────┐
    │                LAYER 4 - TRANSPORT LAYER                 │
    └──────────────────────────────────────────────────────────┘
```

```
┌──────────────────────────────────────────────────────────┐
│  PDU: SEGMENT (TCP) / DATAGRAM (UDP)                     │
└──────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────┐
│  1. SERVICE POINT ADDRESSING                             │
│     • Port numbers (0-65535)                             │
│     • Identifies applications                            │
│     • Socket = IP + Port                                 │
└──────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────┐
│  2. SEGMENTATION & REASSEMBLY                            │
│     • Breaks large data into segments                    │
│     • Reassembles at destination                         │
│     • Uses sequence numbers                              │
└──────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────┐
│  3. FLOW CONTROL                                         │
│     • Controls transmission speed                        │
│     • Sliding window protocol                            │
│     • Prevents receiver overflow                         │
└──────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────┐
│  4. ERROR CONTROL                                        │
│     • Checksum for error detection                       │
│     • ACK for confirmation                               │
│     • Retransmission if error                            │
└──────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────┐
│  5. CONNECTION CONTROL                                   │
│     • TCP: Connection-oriented (3-way handshake)         │
│     • UDP: Connectionless (no handshake)                 │
└──────────────────────────────────────────────────────────┘
```

Segmentation & Reassembly

```
+-------------------------------------------------------------+
|              SEGMENTATION & REASSEMBLY                      |
+-------------------------------------------------------------+
```

```
|                                                          |
|   SEGMENTATION:                                          |
|   Breaking large data into smaller pieces (segments)     |
|                                                          |
|   REASSEMBLY:                                            |
|   Putting segments back together at destination          |
|                                                          |
+----------------------------------------------------------+

+----------------------------------------------------------+
|                  WHY SEGMENTATION?                        |
+----------------------------------------------------------+
|                                                          |
|   1. NETWORK LIMITS                                      |
|      └── Networks have maximum packet size (MTU)         |
|      └── MTU = Maximum Transmission Unit (usually 1500)  |
|                                                          |
|   2. EFFICIENT TRANSMISSION                              |
|      └── Smaller pieces easier to send                   |
|      └── Less data to resend if error occurs             |
|                                                          |
|   3. FAIR SHARING                                        |
|      └── Multiple users can share network                |
|      └── No single user blocks the network               |
|                                                          |
+----------------------------------------------------------+
```

Service Point Addressing (Port Numbers)

```
+----------------------------------------------------------+
|            SERVICE POINT ADDRESSING                       |
+----------------------------------------------------------+
|                                                          |
|   WHAT: Uses PORT NUMBERS to identify applications       |
|                                                          |
|   WHY:                                                   |
|   One computer can run multiple applications             |
|   Port numbers tell which app should receive data        |
|                                                          |
+----------------------------------------------------------+

   EXAMPLE:
```

```
┌──────────────────────────────────────────────────────┐
│                                                      │
│   Your Computer (192.168.1.10)                       │
│                                                      │
│   ┌──────────────┐  ┌──────────────┐  ┌──────────────┐ │
│   │   Chrome     │  │   Email      │  │   Game       │ │
│   │  (Port 443)  │  │ (Port 25)    │  │ (Port 3074)  │ │
│   └──────────────┘  └──────────────┘  └──────────────┘ │
│                                                      │
│   How does computer know which app gets the data?    │
│   ANSWER: PORT NUMBER!                               │
│                                                      │
└──────────────────────────────────────────────────────┘


DATA ARRIVES:
┌──────────────────────────────────────────────────────┐
│                                                      │
│   Destination IP: 192.168.1.10                       │
│   Destination Port: 443                              │
│                                                      │
│   Computer says: "Port 443? That's for Chrome!"      │
│   Data sent to → Chrome browser                      │
│                                                      │
└──────────────────────────────────────────────────────┘
```

Flow Control

```
+-------------------------------------------------------+
|                     FLOW CONTROL                      |
+-------------------------------------------------------+
|                                                       |
|   WHAT:                                               |
|   Controls the SPEED of data transmission             |
|   Prevents sender from overwhelming receiver          |
|                                                       |
|   WHY:                                                |
|   • Sender may be faster than receiver                |
|   • Receiver buffer may get full                      |
|   • Prevents data loss                                |
|                                                       |
+-------------------------------------------------------+
```

Error Control

```
+-----------------------------------------------------------+
|                      ERROR CONTROL                         |
+-----------------------------------------------------------+
|                                                           |
|   WHAT:                                                   |
|   Detects and corrects errors in transmission             |
|                                                           |
|   METHODS:                                                |
|   • Checksum (detect errors)                              |
|   • Acknowledgment (confirm receipt)                      |
|   • Retransmission (resend if error)                      |
|                                                           |
+-----------------------------------------------------------+
```

Connection Control

```
+-----------------------------------------------------------+
|                    CONNECTION CONTROL                      |
+-----------------------------------------------------------+
|                                                           |
|   WHAT:                                                   |
|   Manages the connection between sender and receiver      |
|                                                           |
|   TWO TYPES:                                              |
|   • Connection-Oriented (TCP)                             |
|   • Connectionless (UDP)                                  |
|                                                           |
+-----------------------------------------------------------+
```

TCP VS UDP

```
┌──────────────────────────────────────────────────────────────────┐
│                      TCP vs UDP                                  │
├──────────────────────────┬──────────────────────────────────────┤
│         TCP              │               UDP                    │
├──────────────────────────┼──────────────────────────────────────┤
│  Connection-oriented     │  Connectionless                      │
│  Reliable                │  Unreliable (Best-effort)            │
│  Ordered delivery        │  No ordering guarantee               │
```

```
│  Error checking + Fix  │  Error checking only                  │
│  Flow control (Yes)    │  Flow control (No)                    │
│  Slower                │  Faster                               │
│  Larger header (20B)   │  Smaller header (8B)                  │
│                        │                                       │
│  Use Cases:            │  Use Cases:                           │
│  • HTTP/HTTPS          │  • DNS queries                        │
│  • Email (SMTP/POP3)   │  • VoIP                               │
│  • File Transfer (FTP) │  • Video streaming                    │
│  • SSH                 │  • Online gaming                      │
│  • Database            │  • DHCP                               │
└────────────────────────┴───────────────────────────────────────┘
```

PDU - PROTOCOL DATA UNIT

```
+-----------------------------------------------------------+
|                 PDU - PROTOCOL DATA UNIT                  |
+-----------------------------------------------------------+
|                                                           |
|   PDU = Data format at each OSI layer                     |
|                                                           |
|   Each layer has its own PDU name:                        |
|                                                           |
|   ┌──────────────────────────────────────────────────┐    |
|   │  Layer 7, 6, 5  │  DATA                          │    |
|   ├─────────────────┼────────────────────────────────┤    |
|   │  Layer 4        │  SEGMENT (TCP) / DATAGRAM (UDP)│    |
|   ├─────────────────┼────────────────────────────────┤    |
|   │  Layer 3        │  PACKET                        │    |
|   ├─────────────────┼────────────────────────────────┤    |
|   │  Layer 2        │  FRAME                         │    |
|   ├─────────────────┼────────────────────────────────┤    |
|   │  Layer 1        │  BITS                          │    |
|   └─────────────────┴────────────────────────────────┘    |
|                                                           |
+-----------------------------------------------------------+
```

```
+-----------------------------------------------------------+
|                    PDU AT EACH LAYER                      |
+-----------------------------------------------------------+

   Layer 7 ──────────>  ┌──────────────────────┐
```

```
Layer 6 ──────────>  │        DATA          │
Layer 5 ──────────>  └──────────────────────┘
                              │
                              ▼
Layer 4 ──────────>  ┌─────────┬──────────────┐
                     │TCP/UDP  │    DATA      │
                     │Header   │              │
                     └─────────┴──────────────┘

                           SEGMENT
                              │
                              ▼
Layer 3 ──────────>  ┌─────────┬─────────┬──────────────┐
                     │   IP    │TCP/UDP  │    DATA      │
                     │ Header  │Header   │              │
                     └─────────┴─────────┴──────────────┘

                            PACKET
                              │
                              ▼
Layer 2 ──────────>  ┌────────┬────────┬────────┬──────┬───────┐
                     │Ethernet│   IP   │TCP/UDP │ DATA │Trailer│
                     │ Header │ Header │Header  │      │       │
                     └────────┴────────┴────────┴──────┴───────┘

                                   FRAME
                              │
                              ▼
Layer 1 ──────────>  01101001 01101110 01100110 01101111...
                                   BITS
```

TCP 3 way handshake & 4 way handshake

```
  CLIENT                                          SERVER
     │                                               │
     │        1. SYN (seq=100)                       │
     │ ──────────────────────────────────────────────►│
     │        "I want to connect"                    │
     │                                               │
     │        2. SYN-ACK (seq=300, ack=101)          │
     │ ◄──────────────────────────────────────────────│
     │        "OK, I acknowledge"                    │
     │                                               │
     │        3. ACK (seq=101, ack=301)              │
     │ ──────────────────────────────────────────────►│
     │        "Connection established"               │
```

```
        │                                                │
        │  ═══════ CONNECTION ESTABLISHED ═══════        │
        │                                                │


TCP CONNECTION TERMINATION (Four-Way Handshake):
═══════════════════════════════════════════════════════════════

     CLIENT                                          SERVER
        │                                                │
        │        1. FIN                                  │
        │ ──────────────────────────────────────────────► │
        │                                                │
        │        2. ACK                                  │
        │ ◄────────────────────────────────────────────── │
        │                                                │
        │        3. FIN                                  │
        │ ◄────────────────────────────────────────────── │
        │                                                │
        │        4. ACK                                  │
        │ ──────────────────────────────────────────────► │
        │                                                │
        │  ═══════ CONNECTION CLOSED ═══════════         │
```

More at [09. TCP & UDP]

Layer 3: Network Layer

Routing & Logical Addressing (IP)
"Packet" layer – decides the best path across networks.

- IP (IPv4 / IPv6)
- ICMP (ping, traceroute)
- IPsec (can operate here in transport/tunnel mode)
- Routing protocols: **OSPF**, BGP, EIGRP RIP

```
┌──────────────────────────────────────────────────────────────────┐
│                        NETWORK LAYER                             │
├──────────────────────────────────────────────────────────────────┤
│  PDU: PACKET                                                     │
│  Devices: Routers, Layer 3 Switches                             │
│  Addressing: IP Addresses (IPv4/IPv6)                           │
```

```
│  Functions:                                                    │
│    • Logical Addressing                                        │
│    • Routing (Path Determination)                              │
│    • Packet Forwarding                                         │
│    • Fragmentation and Reassembly                             │
│    • Error Handling (ICMP)                                     │
└────────────────────────────────────────────────────────────────┘
```

```
+-----------------------------------------------------------+
|               LAYER 3 - 5 MAIN FUNCTIONS                  |
+-----------------------------------------------------------+
|                                                           |
|   1. LOGICAL ADDRESSING (IP Addressing)                   |
|                                                           |
|   2. ROUTING (Path Determination)                         |
|                                                           |
|   3. FRAGMENTATION & REASSEMBLY                           |
|                                                           |
|   4. PACKET FORWARDING                                    |
|                                                           |
|   5. ERROR HANDLING & DIAGNOSTICS                         |
|                                                           |
+-----------------------------------------------------------+
```

Logical Addressing (IP Addressing)

```
+-----------------------------------------------------------+
|                 LOGICAL ADDRESSING                        |
+-----------------------------------------------------------+
|                                                           |
|   WHAT:                                                   |
|   Assigns unique IP address to every device               |
|                                                           |
|   WHY:                                                    |
|   MAC addresses work only in local network                |
|   IP addresses work across ALL networks (global)          |
|                                                           |
|   TYPES:                                                  |
|   ┌──────────────────────────────────────────────────┐    |
|   │  IPv4: 32-bit address                            │    |
|   │        Example: 192.168.1.10                     │    |
|   │        Format: X.X.X.X (0-255 each)              │    |
```

```
|   |                                                   |    |
|   |  IPv6: 128-bit address                            |    |
|   |        Example: 2001:0db8:85a3::8a2e:0370:7334  |    |
|   |        Format: X:X:X:X:X:X:X:X (hex)            |    |
|   └──────────────────────────────────────────────────┘    |
|                                                           |
+-----------------------------------------------------------+
```

Routing (Path Determination)

```
+-----------------------------------------------------------+
|                      ROUTING                              |
+-----------------------------------------------------------+
|                                                           |
|   WHAT:                                                   |
|   Finding the BEST PATH from source to destination        |
|                                                           |
|   HOW:                                                    |
|   Routers use ROUTING TABLES to make decisions            |
|                                                           |
+-----------------------------------------------------------+

   EXAMPLE:
   ┌──────────────────────────────────────────────────────┐
   |                                                      |
   |   Source: 192.168.1.10 (Your PC)                     |
   |   Destination: 142.250.192.46 (Google)               |
   |                                                      |
   |   Your PC → Router 1 → Router 2 → Router 3 → Google |
   |                                                      |
   |   Each router decides: "Where to send next?"         |
   |                                                      |
   └──────────────────────────────────────────────────────┘
```

Fragmentation & Reassembly

```
+-----------------------------------------------------------+
|         LAYER 4 vs LAYER 3 FRAGMENTATION                  |
+-----------------------------------------------------------+

+--------------------------------+--------------------------------+
|     LAYER 4 (Transport)        |     LAYER 3 (Network)          |
```

```
|     SEGMENTATION                |     FRAGMENTATION               |
+---------------------------------+---------------------------------+
|                                 |                                 |
| WHAT:                           | WHAT:                           |
| Breaks APPLICATION DATA         | Breaks PACKETS                  |
| into SEGMENTS                   | into FRAGMENTS                  |
|                                 |                                 |
+---------------------------------+---------------------------------+
|                                 |                                 |
| WHY:                            | WHY:                            |
| Application data too large      | Packet larger than              |
| for network                     | network's MTU                   |
|                                 |                                 |
+---------------------------------+---------------------------------+
|                                 |                                 |
| WHEN:                           | WHEN:                           |
| BEFORE sending                  | DURING transmission             |
| (at source)                     | (at any router)                 |
|                                 |                                 |
+---------------------------------+---------------------------------+
|                                 |                                 |
| WHERE:                          | WHERE:                          |
| Only at SOURCE                  | At SOURCE or any ROUTER         |
| (sender's computer)             | along the path                  |
|                                 |                                 |
+---------------------------------+---------------------------------+
|                                 |                                 |
| WHO REASSEMBLES:                | WHO REASSEMBLES:                |
| Destination (Layer 4)           | Destination (Layer 3)           |
|                                 |                                 |
+---------------------------------+---------------------------------+
|                                 |                                 |
| BASED ON:                       | BASED ON:                       |
| MSS (Maximum Segment Size)      | MTU (Maximum Transmission Unit)|
| Negotiated during handshake     | Each network's limit            |
|                                 |                                 |
+---------------------------------+---------------------------------+
```

MSS vs MTU

```
+------------------------------------------------------------+
|                      MSS vs MTU                            |
+------------------------------------------------------------+
```

```
MTU (Maximum Transmission Unit):
┌──────────────────────────────────────────────────────┐
│                                                      │
│   • Layer 3 concept                                  │
│   • Maximum size of ENTIRE PACKET                    │
│   • Includes headers + data                          │
│   • Set by each network/link                         │
│   • Ethernet MTU = 1500 bytes (typical)              │
│                                                      │
└──────────────────────────────────────────────────────┘

MSS (Maximum Segment Size):
┌──────────────────────────────────────────────────────┐
│                                                      │
│   • Layer 4 concept (TCP only)                       │
│   • Maximum size of DATA in segment                  │
│   • Does NOT include headers                         │
│   • Negotiated during TCP handshake                  │
│   • MSS = MTU - IP Header - TCP Header               │
│   • MSS = 1500 - 20 - 20 = 1460 bytes (typical)      │
│                                                      │
└──────────────────────────────────────────────────────┘

RELATIONSHIP:
┌──────────────────────────────────────────────────────┐
│                                                      │
│   ┌──────────────────── MTU (1500) ──────────────────┐ │
│   │                                                  │ │
│   │  IP Header  │  TCP Header  │      DATA         │ │
│   │   (20 B)    │    (20 B)    │   (MSS = 1460)    │ │
│   │                                                  │ │
│   └──────────────────────────────────────────────────┘ │
│                                                      │
└──────────────────────────────────────────────────────┘
```

Packet Forwarding

```
+-------------------------------------------------------+
|                   PACKET FORWARDING                    |
+-------------------------------------------------------+
|                                                       |
|   WHAT:                                               |
```

```
|   Moving packets from one network to another             |
|                                                          |
|   HOW:                                                   |
|   Router examines destination IP and forwards            |
|   to next hop                                            |
|                                                          |
+----------------------------------------------------------+

   PROCESS:
   ┌──────────────────────────────────────────────────────┐
   │                                                      │
   │  1. Packet arrives at router                         │
   │                    ▼                                 │
   │  2. Router reads DESTINATION IP address              │
   │                    ▼                                 │
   │  3. Router checks ROUTING TABLE                      │
   │                    ▼                                 │
   │  4. Router finds NEXT HOP                            │
   │                    ▼                                 │
   │  5. Router forwards packet to next hop               │
   │                                                      │
   └──────────────────────────────────────────────────────┘
```

Error Handling & Diagnostics

```
+----------------------------------------------------------+
|             ERROR HANDLING & DIAGNOSTICS                  |
+----------------------------------------------------------+
|                                                          |
|   PROTOCOL: ICMP (Internet Control Message Protocol)     |
|                                                          |
|   FUNCTIONS:                                             |
|   ┌──────────────────────────────────────────────┐       |
|   │                                              │       |
|   │  • Ping (Echo Request/Reply)                 │       |
|   │  • Destination Unreachable                   │       |
|   │  • Time Exceeded (TTL = 0)                   │       |
|   │  • Redirect (better route available)         │       |
|   │  • Traceroute                                │       |
|   │                                              │       |
|   └──────────────────────────────────────────────┘       |
```

```
|                                                           |
+-----------------------------------------------------------+
```

IPv4 Header Structure

```
+-----------------------------------------------------------+
|                    IPv4 HEADER FORMAT                     |
+-----------------------------------------------------------+

   ←──────────────────── 32 bits (4 bytes) ──────────────────→

    0                   1                   2                   3
    0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
   +-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
   |Version|  IHL  |    DSCP   |ECN|          Total Length         |
   | (4)   | (4)   |    (6)    |(2)|            (16)               |
   +-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+  ← 4 bytes
   |         Identification        |Flags|      Fragment Offset    |
   |            (16)               | (3) |          (13)           |
   +-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+  ← 4 bytes
   |  Time to Live |    Protocol   |         Header Checksum       |
   |     (8)       |      (8)      |            (16)               |
   +-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+  ← 4 bytes
   |                                                               |
   |                       Source IP Address                       |  ← 4 bytes
   |                            (32)                               |
   +-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
   |                                                               |
   |                    Destination IP Address                     |  ← 4 bytes
   |                            (32)                               |
   +-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
   |                                                               |
   |                    Options (if IHL > 5)                       |  ← 0-40
bytes
   |                        (Variable)                             |
   +-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
   |                                                               |
   |                           DATA                                |  ←
Variable
   |                    (TCP/UDP Segment)                          |
   |                                                               |
   +-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
```

```
HEADER SIZE: 20 bytes (minimum) to 60 bytes (maximum)
TOTAL FIELDS: 12 (without options)
```

S.No	Field Name	Size	Purpose / Description	Example
1	Version	4 bits	IP version number	4 (IPv4), 6 (IPv6)
2	IHL (Header Length)	4 bits	Header length in 32-bit words. Min: 5 (20 bytes), Max: 15 (60 bytes)	5 = 20 bytes
3	DSCP	6 bits	Differentiated Services Code Point. Quality of Service (QoS)	Priority marking
4	ECN	2 bits	Explicit Congestion Notification. Network congestion indicator	00 = not used
5	Total Length	16 bits	Total packet size (header + data). Max: 65535 bytes	1500 bytes
6	Identification	16 bits	Unique ID for each packet. Used to reassemble fragments	12345
7	Flags	3 bits	Controls fragmentation. Bit 0: Reserved, Bit 1: DF (Don't Fragment), Bit 2: MF (More Fragments)	DF=1 (don't fragment)
8	Fragment Offset	13 bits	Position of fragment in original packet. Measured in 8-byte units	0 = first fragment
9	TTL (Time to Live)	8 bits	Maximum hops before packet is discarded. Decremented by each router	64, 128, 255
10	Protocol	8 bits	Upper layer protocol. TCP=6, UDP=17, ICMP=1	6 (TCP)
11	Header Checksum	16 bits	Error detection for header ONLY (not data)	Calculated value
12	Source IP	32 bits	Sender's IP address	192.168.1.10
13	Destination IP	32 bits	Receiver's IP address	142.250.192.46
14	Options	0-40 bytes	Optional features (rarely used). Timestamp, Record Route, etc.	Usually not present
15	Data	Variable	Payload from upper layer (TCP/UDP segment)	TCP segment

Layer 2: Data Link Layer

```
┌──────────────────────────────────────────────────────────────────────┐
│                        DATA LINK LAYER                               │
├──────────────────────────────────────────────────────────────────────┤
│  PDU: FRAME                                                          │
│  Devices: Switches, Bridges, NICs                                   │
│  Addressing: MAC Addresses (48-bit)                                 │
│  Functions:                                                          │
│    • Physical Addressing (MAC)                                       │
│    • Frame Creation and Detection                                   │
│    • Media Access Control                                            │
│    • Flow Control                                                    │
└──────────────────────────────────────────────────────────────────────┘
```

```
+-----------------------------------------------------------+
|                 LAYER 2 SUB-LAYERS                        |
+-----------------------------------------------------------+
|                                                           |
|   Layer 2 is divided into TWO sub-layers:                 |
|                                                           |
|   ┌──────────────────────────────────────────────────┐    |
|   │                                                  │    |
|   │   ┌──────────────────────────────────────────┐   │    |
|   │   │         LLC (Logical Link Control)       │   │    |
|   │   │              Upper Sub-Layer             │   │    |
|   │   └──────────────────────────────────────────┘   │    |
|   │                                                  │    |
|   │   ┌──────────────────────────────────────────┐   │    |
|   │   │         MAC (Media Access Control)       │   │    |
|   │   │              Lower Sub-Layer             │   │    |
|   │   └──────────────────────────────────────────┘   │    |
|   │                                                  │    |
|   └──────────────────────────────────────────────────┘    |
|                                                           |
+-----------------------------------------------------------+
```

LLC vs MAC

```
+-----------------------------------------------------------+
|                     LLC vs MAC                            |
```

```
+-------------------------------------------------------------+

+--------------------------------+--------------------------------+
|       LLC (Upper Sub-Layer)    |       MAC (Lower Sub-Layer)    |
+--------------------------------+--------------------------------+
|                                |                                |
| FULL NAME:                     | FULL NAME:                     |
| Logical Link Control           | Media Access Control           |
|                                |                                |
+--------------------------------+--------------------------------+
|                                |                                |
| STANDARD:                      | STANDARD:                      |
| IEEE 802.2                     | IEEE 802.3 (Ethernet)          |
|                                | IEEE 802.11 (WiFi)             |
|                                |                                |
+--------------------------------+--------------------------------+
|                                |                                |
| FUNCTIONS:                     | FUNCTIONS:                     |
| • Interface with Layer 3       | • Physical addressing (MAC)    |
| • Flow control                 | • Media access control         |
| • Error control                | • Frame delimiting             |
| • Protocol multiplexing        | • Error detection (FCS)        |
|                                |                                |
+--------------------------------+--------------------------------+
|                                |                                |
| IDENTIFIES:                    | IDENTIFIES:                    |
| Upper layer protocol           | Devices on network             |
| (IP, IPX, AppleTalk)           | (Source & Destination MAC)     |
|                                |                                |
+--------------------------------+--------------------------------+
|                                |                                |
| EXAMPLE:                       | EXAMPLE:                       |
| "This frame contains IP data"  | "Send to AA:BB:CC:DD:EE:FF"    |
|                                |                                |
+--------------------------------+--------------------------------+
```

Functions

```
+-------------------------------------------------------------+
|              LAYER 2 - MAIN FUNCTIONS                        |
+-------------------------------------------------------------+
|                                                             |
|   1. PHYSICAL ADDRESSING (MAC Address)                      |
```

```
|                                                          |
|   2. FRAMING                                             |
|                                                          |
|   3. MEDIA ACCESS CONTROL                                |
|                                                          |
|   4. ERROR DETECTION                                     |
|                                                          |
|   5. FLOW CONTROL                                        |
|                                                          |
+----------------------------------------------------------+
```

Framing

```
+----------------------------------------------------------+
|                       FRAMING                             |
+----------------------------------------------------------+
|                                                          |
|   WHAT:                                                  |
|   Encapsulating network layer packet into a FRAME        |
|   Adding header and trailer to packet                    |
|                                                          |
|   WHY:                                                   |
|   • Defines where frame starts and ends                  |
|   • Adds addressing (MAC)                                |
|   • Adds error detection (FCS)                           |
|                                                          |
+----------------------------------------------------------+

   FRAMING PROCESS:
   ┌──────────────────────────────────────────────────────┐
   │                                                      │
   │   From Layer 3:                                      │
   │   ┌──────────────────────────────────────────────┐   │
   │   │              IP PACKET                       │   │
   │   └──────────────────────────────────────────────┘   │
   │                                                      │
   │                       ▼                              │
   │                                                      │
   │   Layer 2 adds Header and Trailer:                   │
   │   ┌────────┬──────────────────────────┬─────────┐    │
   │   │ HEADER │       IP PACKET          │TRAILER  │    │
   │   │(14-22B)│                          │  (4B)   │    │
   │   └────────┴──────────────────────────┴─────────┘    │
   │                                                      │
```

```
    │                                                          │
    │                    = FRAME                               │
    │                                                          │
    └──────────────────────────────────────────────────────────┘
```

Media Access Control

```
+-------------------------------------------------------------+
|                 MEDIA ACCESS CONTROL                         |
+-------------------------------------------------------------+
|                                                              |
|   WHAT:                                                      |
|   Controls HOW devices access the shared medium              |
|   Prevents collisions when multiple devices send             |
|                                                              |
|   METHODS:                                                   |
|    ┌──────────────────────────────────────────────────┐      |
|    │                                                  │      |
|    │  1. CSMA/CD (Carrier Sense Multiple Access      │      |
|    │              with Collision Detection)           │      |
|    │     → Used in: Wired Ethernet (Half-duplex)      │      |
|    │                                                  │      |
|    │  2. CSMA/CA (Carrier Sense Multiple Access      │      |
|    │              with Collision Avoidance)           │      |
|    │     → Used in: Wireless (WiFi)                   │      |
|    │                                                  │      |
|    │  3. Token Passing                                │      |
|    │     → Used in: Token Ring, FDDI                  │      |
|    │                                                  │      |
|    └──────────────────────────────────────────────────┘      |
|                                                              |
+-------------------------------------------------------------+
```

Error Detection

```
+-------------------------------------------------------------+
|                    ERROR DETECTION                           |
+-------------------------------------------------------------+
|                                                              |
|   WHAT:                                                      |
|   Detecting errors that occurred during transmission         |
|                                                              |
```

```
|   METHOD:                                                |
|   FCS (Frame Check Sequence) using CRC                   |
|   (Cyclic Redundancy Check)                              |
|                                                          |
|   NOTE:                                                  |
|   Layer 2 only DETECTS errors                            |
|   It does NOT CORRECT them                               |
|   If error found, frame is simply DISCARDED              |
|                                                          |
+----------------------------------------------------------+
```

Flow Control

```
+----------------------------------------------------------+
|                    FLOW CONTROL                           |
+----------------------------------------------------------+
|                                                          |
|   WHAT:                                                  |
|   Controls the rate of data transmission                 |
|   Prevents fast sender from overwhelming slow receiver   |
|                                                          |
|   NOTE:                                                  |
|   Layer 2 flow control is less common today              |
|   Usually handled by Layer 4 (TCP)                       |
|                                                          |
|   METHODS (when used):                                   |
|   ┌──────────────────────────────────────────────────┐   |
|   │                                                  │   |
|   │  1. PAUSE Frame (IEEE 802.3x)                    │   |
|   │     → Used in Full-duplex Ethernet               │   |
|   │     → Receiver sends PAUSE to slow down sender   │   |
|   │                                                  │   |
|   │  2. Priority Flow Control (PFC)                  │   |
|   │     → Used in Data Center networks               │   |
|   │     → Per-priority PAUSE                         │   |
|   │                                                  │   |
|   └──────────────────────────────────────────────────┘   |
|                                                          |
+----------------------------------------------------------+.
```

Layer 1: Physical Layer

```
╔═══════════════════════════════════════════════════════════════════════╗
║                     LAYER 1: PHYSICAL LAYER                           ║
║                     "The Electrical Layer"                            ║
╚═══════════════════════════════════════════════════════════════════════╝
╔═══════════════════════════════════════════════════════════════════════╗
║  Data Unit: BITS (0s and 1s)                                          ║
║  Function: Transmit raw binary data over physical medium              ║
╚═══════════════════════════════════════════════════════════════════════╝
```

```
TRANSMISSION TYPES:
├── Electrical signals (Copper cables)
├── Light pulses (Fiber optic)
└── Radio waves (Wireless)

TRANSMISSION MODES:
┌──────────────┬──────────────────────────────────────────┐
│ Simplex      │ One direction only      →                │
│ Half-Duplex  │ Both directions, one    ↔ (alternating)│
│              │ at a time                                │
│ Full-Duplex  │ Both directions         ⇔ (simultaneous)│
│              │ simultaneously                           │
└──────────────┴──────────────────────────────────────────┘
```

Physical Media Types

```
GUIDED MEDIA (Wired):
═══════════════════════════════════════════════════════════════════
│ Type               │ Speed        │ Distance │ Use Case      │
═══════════════════════════════════════════════════════════════════
│ Coaxial Cable      │ 10 Mbps      │ 500m     │ Legacy, CATV  │
│ UTP Cat5e          │ 1 Gbps       │ 100m     │ Office LAN    │
│ UTP Cat6           │ 10 Gbps      │ 55m      │ Data centers  │
│ UTP Cat6a          │ 10 Gbps      │ 100m     │ High-speed LAN│
│ UTP Cat7           │ 10 Gbps      │ 100m     │ Enterprise    │
│ UTP Cat8           │ 40 Gbps      │ 30m      │ Data centers  │
│ Single-mode Fiber  │ 100+ Gbps    │ 100km+   │ WAN, Backbone │
│ Multi-mode Fiber   │ 10 Gbps      │ 550m     │ Campus, SAN   │
═══════════════════════════════════════════════════════════════════

UNGUIDED MEDIA (Wireless):
═══════════════════════════════════════════════════════════════════
│ Type               │ Frequency    │ Speed    │ Range         │
```

```
═══════════════════════════════════════════════════════════
│ Wi-Fi 4 (802.11n) │ 2.4/5 GHz   │ 600 Mbps │ 70m indoor    │
│ Wi-Fi 5 (802.11ac)│ 5 GHz       │ 3.5 Gbps │ 35m indoor    │
│ Wi-Fi 6 (802.11ax)│ 2.4/5/6 GHz │ 9.6 Gbps │ 35m indoor    │
│ Bluetooth         │ 2.4 GHz     │ 3 Mbps   │ 10m           │
│ 5G NR            │ Sub-6/mmWave │ 20 Gbps  │ Varies        │
═══════════════════════════════════════════════════════════
```

Network Devices at Layer 1

```
┌──────────────────────────────────────────────────────────┐
│ HUB                                                      │
│ ═══                                                      │
│ • Broadcasts to ALL ports                                │
│ • No intelligence (dumb device)                          │
│ • Creates single collision domain                        │
│ • Largely obsolete                                       │
│                                                          │
│          ┌───┐                                           │
│     PC──┤   ├──PC                                        │
│         │HUB│                                            │
│     PC──┤   ├──PC                                        │
│          └───┘                                           │
│   (All devices receive all traffic)                      │
└──────────────────────────────────────────────────────────┘


┌──────────────────────────────────────────────────────────┐
│ REPEATER                                                 │
│ ═══════                                                  │
│ • Regenerates/amplifies signals                          │
│ • Extends network distance                               │
│ • No filtering capability                                │
│                                                          │
│   [Signal]──►[REPEATER]──►[Amplified Signal]            │
│   (weak)                  (strong)                       │
└──────────────────────────────────────────────────────────┘


┌──────────────────────────────────────────────────────────┐
│ MODEM (Modulator-Demodulator)                            │
│ ═══                                                      │
│ • Converts digital ↔ analog signals                      │
│ • Used for DSL, Cable, Dial-up                           │
│                                                          │
```

```
│   [Digital]──►[MODEM]──►[Analog]──►Phone Line              │
└──────────────────────────────────────────────────────────────┘
```